

Image Recognition



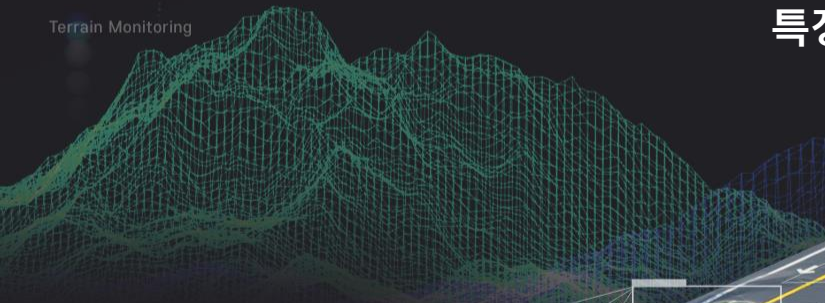
강원혁신플랫폼

컴퓨터 비전



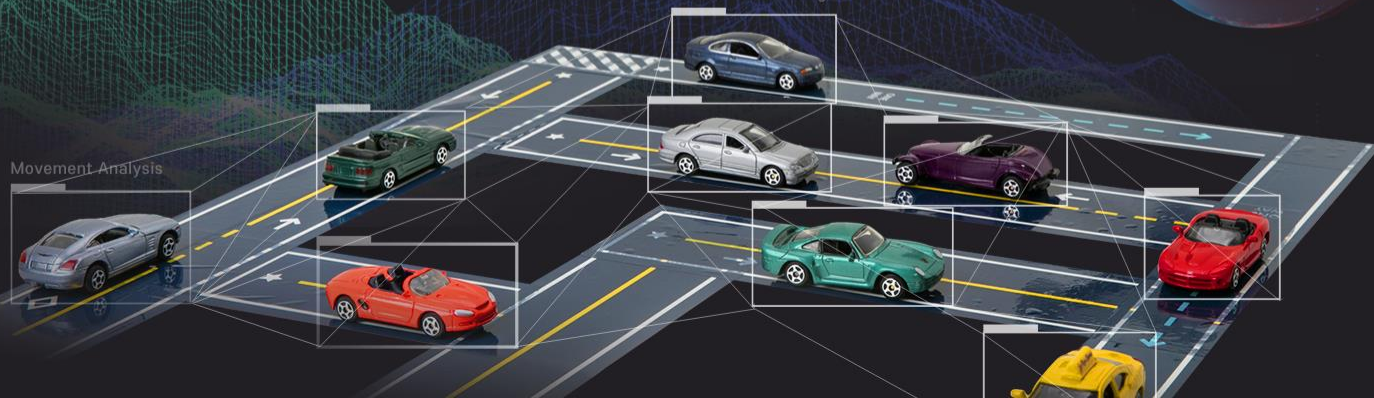
특정 인물 얼굴 인코딩

Terrain Monitoring



Autonomous Driving

Movement Analysis



- 다운로드 받은 이미지들에서 **얼굴의 특징(encoding) 값과 특정한 인물의 이름을 매칭해 딕셔너리(dictionary) 타입으로 저장하는 작업을 진행함**

face_recognition 패키지를 사용하기 위해서는
cmake, dlib 패키지가 먼저 설치되어 있어야 함

- ☞ 또한, dlib 패키지는 C++ 언어로 개발된 라이브러리이기 때문에 이를 컴파일하기 위해서 cmake 패키지도 설치되어 있어야 한다.

cmake



dlib

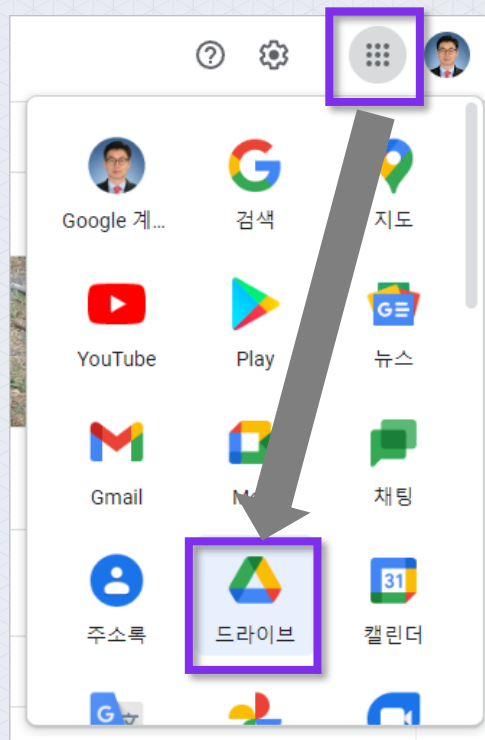


face_recognition

순으로 패키지를 설치

- ☞ 구글 Colab에서는 cmake, dlib 패키지는 기본적으로 설치되어 있어 따로 설치가 필요하지 않다.
- ☞ 하지만, face_recognition 패키지는 구글 Colab에서 기본적으로 지원하지 않아 반드시 설치하여야 함

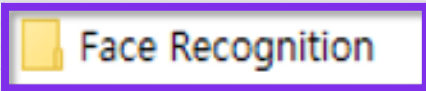
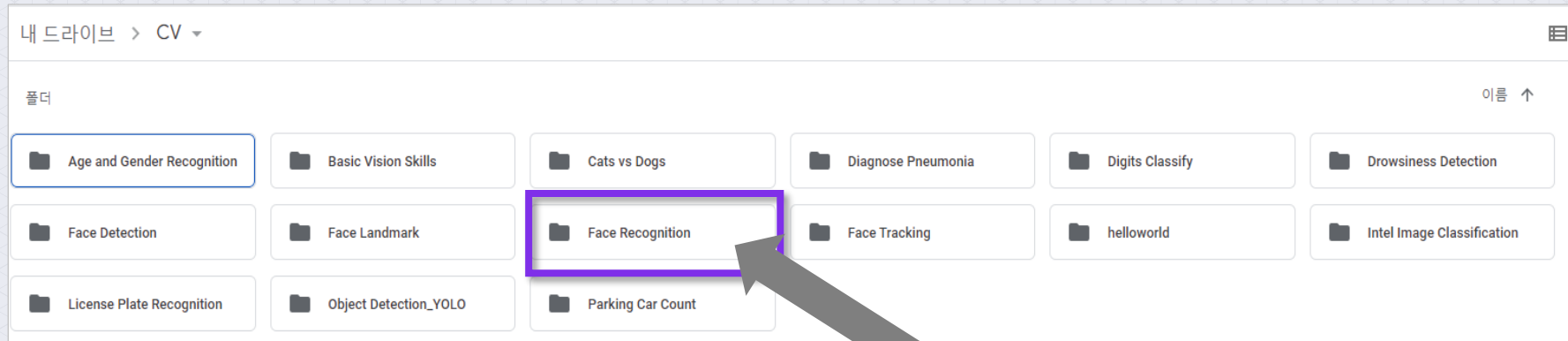
- 먼저, 구글 Colab을 이용해 보자
- 구글 크롬 브라우저에서 아래 그림과 같이 **구글 드라이브**를 클릭



구글 드라이브에서 CV 폴더를 클릭

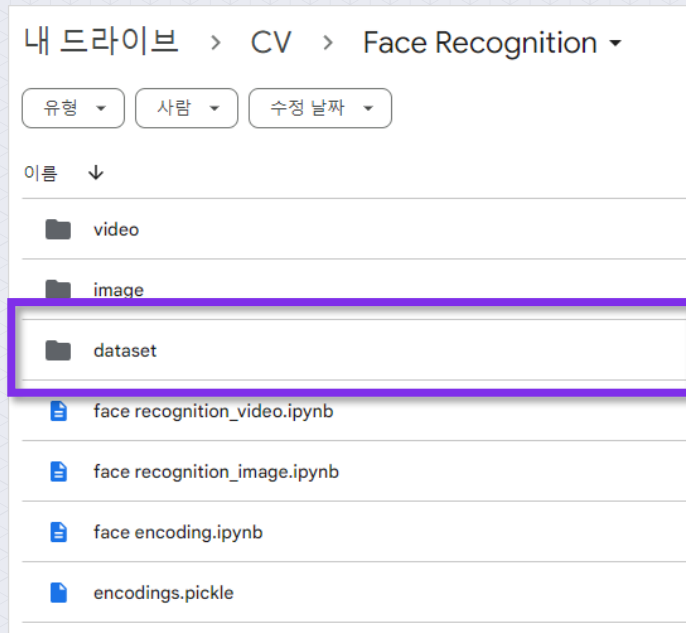
The screenshot shows the Google Drive web interface. On the left sidebar, the '내 드라이브' (My Drive) section is active. Below it, the '폴더' (Folders) section displays a list of folders: 'ipynb_checkpoints', '구글문서' (Google Docs), '구글시트' (Google Sheets), '구글피티' (Google Slides), '스프레드시트' (Spreadsheets), and 'Colab Notebooks'. The 'CV' folder is highlighted with a purple rectangular box. The main content area shows a '추천' (Recommended) section with various image files, including 'Cats vs Dogs.ipynb' and several 'dog.***.jpg' files. The top navigation bar includes the Google Drive logo, a search bar, and user profile icons.

- 구글 드라이브 > CV > Face Recognition 폴더를 클릭
- 강의 실습 자료를 다운로드하여 [Face Recognition] 폴더를 구글 드라이브 CV 폴더 아래에 복사(업로드)

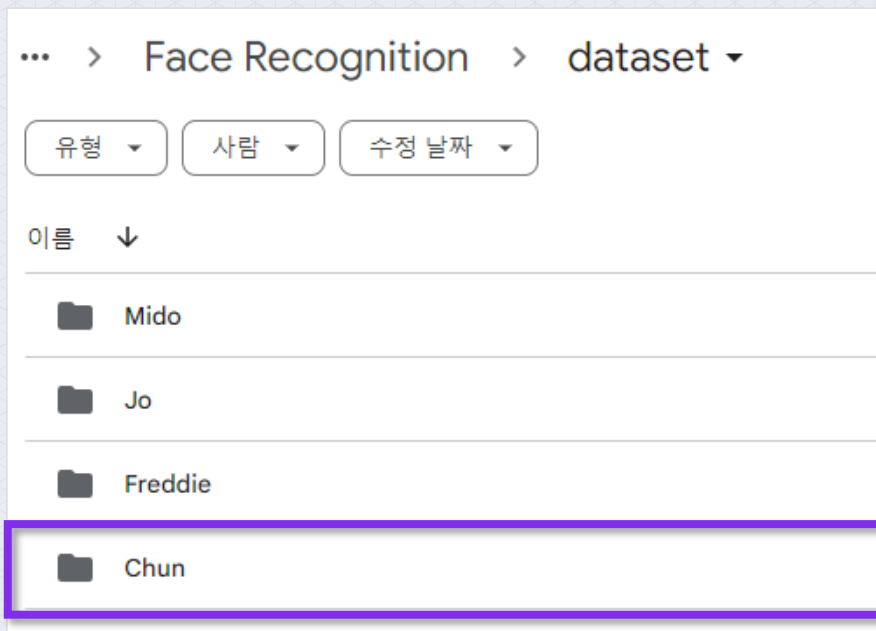


■ 특정 인물 사진 준비하기

- 🔄 구글 드라이브 > CV > Face Recognition > dataset 폴더 클릭
- 🔄 CV > Face Recognition > dataset 폴더 클릭



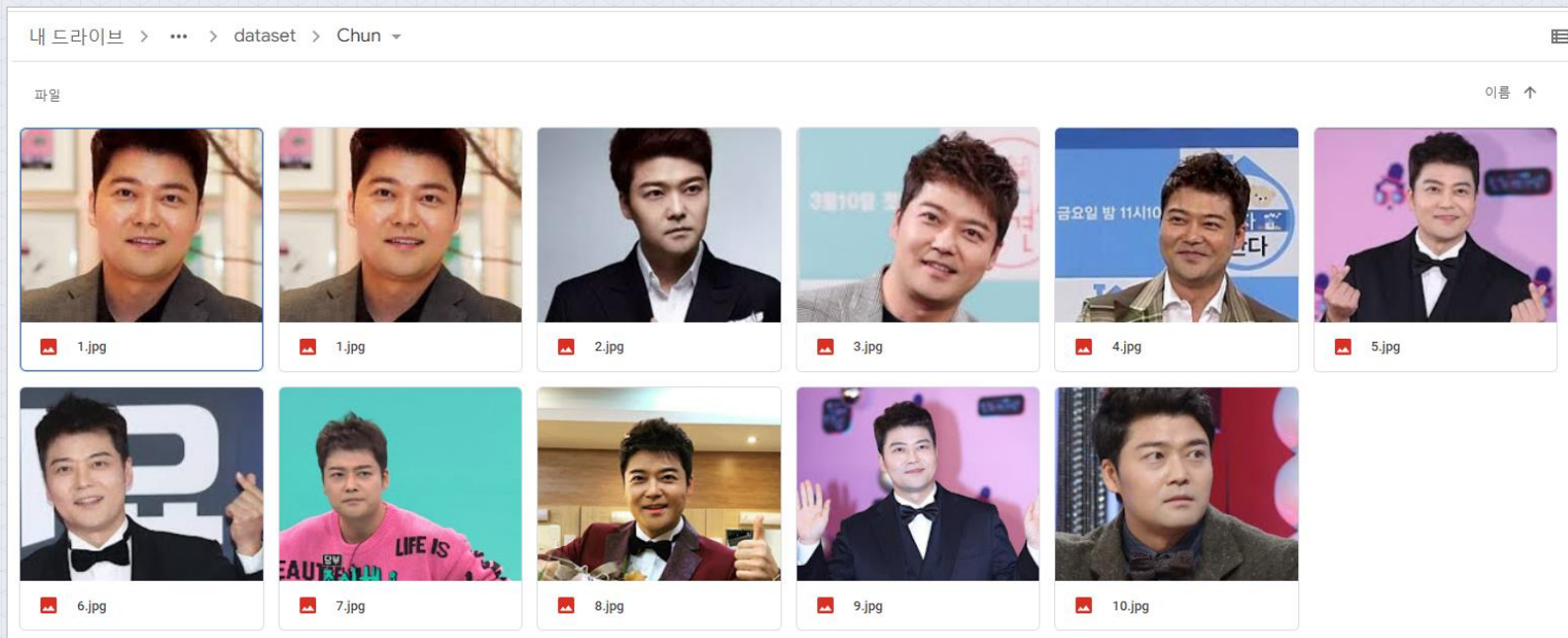
- 구글 드라이브 > CV > Face Recognition > dataset > Chun 폴더 클릭
- 여기서는 전무현 방송인의 얼굴 이미지 사진을 인코딩 한다.
- 특정 인물의 정면 얼굴 이미지 사진 10장을 수집하여 인코딩해보자



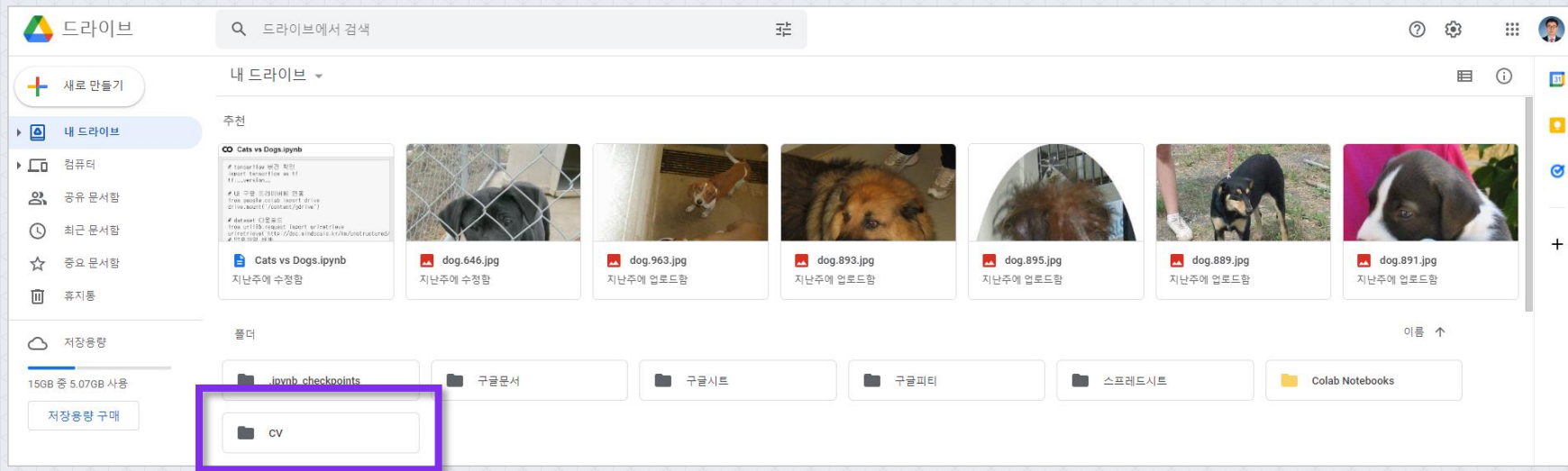
전현무 방송인

■ 전현무 방송인의 얼굴 이미지가 저장된 경로

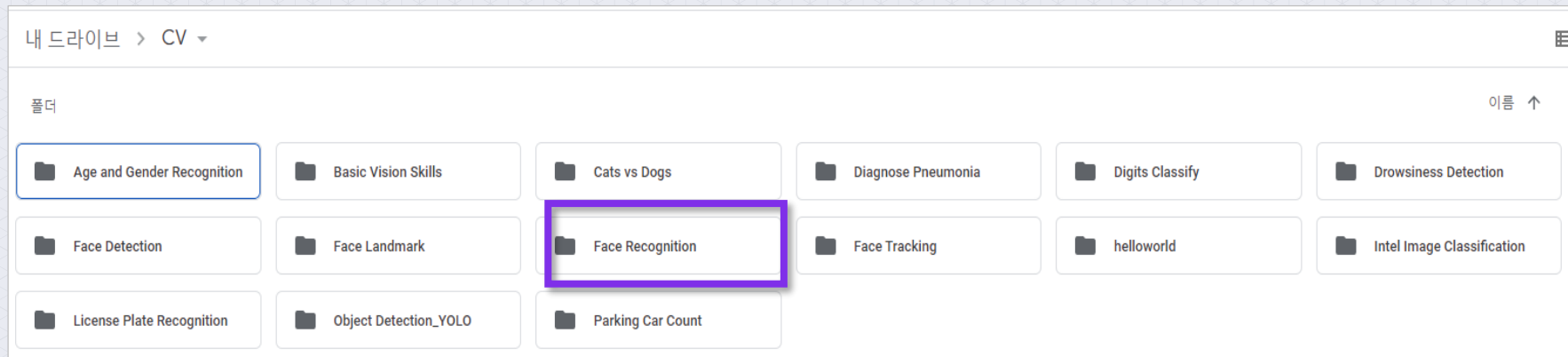
- CV > Face Recognition > dataset > Chun > 1.jpg ~ 10.jpg(총 10장의 이미지)
- 각 이미지에서 생성된 128가지 특성값을 계산함



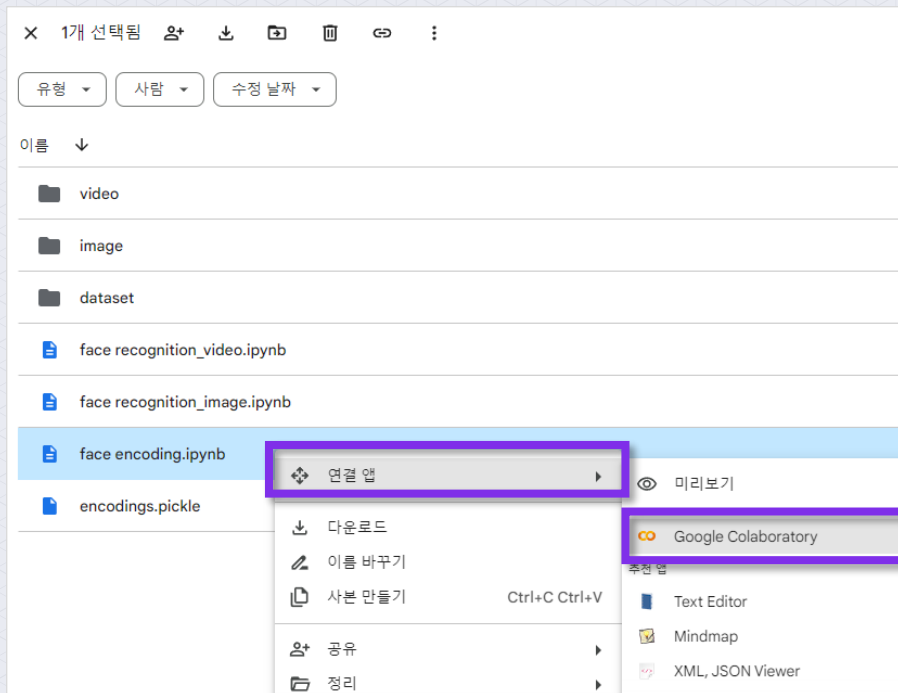
구글 드라이브에서 CV 폴더를 클릭



구글 드라이브 > CV > Face Recognition 폴더를 클릭



- ❶ 아래의 그림의 image 폴더에서 face encoding.ipynb를 선택 후 오른쪽 마우스를 클릭하여 “연결 앱 > Google Colaboratory”을 클릭



🔄 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것임


face encoding.ipynb
☆

파일 수정 보기 삽입 런타임 도구 도움말 모든 변경사항이 저장됨

+ 코드 + 텍스트

☰

🔍

{x}

📁

✓ 27 초

▼ 특정 인물 얼굴 인코딩

▶

```

1 # 내 구글 드라이브에 연동
2
3 from google.colab import drive
4
5 drive.mount('/content/gdrive')

```

📁

Mounted at /content/gdrive

▶

```

1 # 설치된 패키지 확인하기
2
3 !pip list

```

Package	Version
absl-py	1.4.0
aiohttp	3.8.5

✓ 2 초

- ② 내 구글 드라이브에 연동
- ② 구글 Colab 노트북 화면에서 실행 버튼 클릭



- ② [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택하면 gdrive에 연동
- ② gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있음

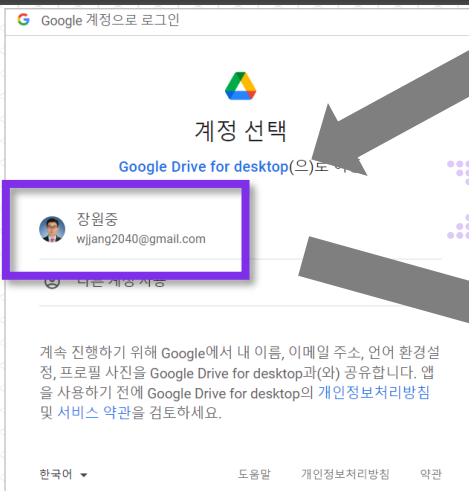
- [Google Drive에 연결] > [계정 선택] > [구글 드라이버 접근 허용]
- 아래 화면을 참고하여 **gdrive**에 연동

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

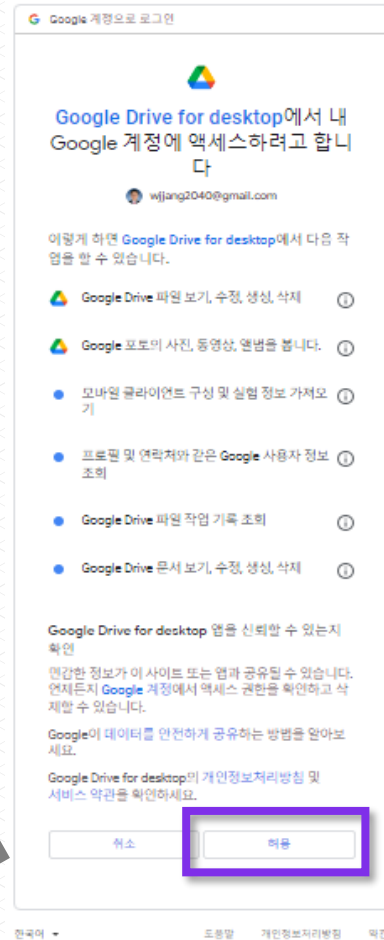
이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니오

Google Drive에 연결



구글 드라이버 접근 허용



■ 구글 Colab에 설치된 패키지 현황을 확인해 보자

🔄 'cmake', 'dlib' 패키지는 이미 설치된 것을 확인할 수 있음

▶	1	# 설치된 패키지 확인하기
	2	
	3	!pip list
📄	cli	0.7.2
	cloudpickle	2.2.1
	cmake	3.25.2
	colorcet	3.0.1
	colorlover	0.3.0
	community	1.0.0b1
	confection	0.1.0
	cons	0.4.6
	contextlib2	21.6.0
	contourpy	1.1.0
	convertdate	2.4.0
	cryptography	3.4.8
	cufflinks	0.17.3
	cupy-cudallx	11.0.0
	cvxopt	1.3.1
	cvxpy	1.3.2
	cycler	0.11.0
	cymem	2.0.7
	Cython	0.29.36
	dask	2022.12.1
	datascience	0.17.6
	db-dtypes	1.1.1
	dbus-python	1.2.18
	debugpy	1.6.6
	decorator	4.4.2
	defusedxml	0.7.1
	distributed	2022.12.1
	dlib	19.24.2
	docutils	0.16
	dopamine-rl	4.0.6
	duckdb	0.8.1

■ face_recognition 패키지의 상세정보 버전을 확인해 보자

- 🔄 확인 결과 'face_recognition' 패키지가 설치되지 않은 것을 볼 수 있음

```
1 # 'face_recognition' 모듈(라이브러리, 패키지)의 상세정보 버전 확인
2
3 !pip show face_recognition
```

```
⚠ WARNING: Package(s) not found: face_recognition
```

■ face_recognition 패키지가 잘 설치된 것을 확인할 수 있음

```
1 # face_recognition 패키지는 구글 Colab에서 기본으로 지원하지 않아 반드시 설치해야 한다.  
2  
3 !pip3 install face_recognition # face_recognition install  
4  
5 # !pip3 uninstall face_recognition # face_recognition uninstall
```

```
Collecting face_recognition  
  Downloading face_recognition-1.3.0-py2.py3-none-any.whl (15 kB)  
Collecting face-recognition-models>=0.3.0 (from face_recognition)  
  Downloading face_recognition_models-0.3.0.tar.gz (100.1 MB)  
----- 100.1/100.1 MB 11.7 MB/s eta 0:00:00  
Preparing metadata (setup.py) ... done  
Requirement already satisfied: Click>=6.0 in /usr/local/lib/python3.10/dist-packages (from face_recognition) (8.1.6)  
Requirement already satisfied: dlib>=19.7 in /usr/local/lib/python3.10/dist-packages (from face_recognition) (19.24.2)  
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from face_recognition) (1.22.4)  
Requirement already satisfied: Pillow in /usr/local/lib/python3.10/dist-packages (from face_recognition) (9.4.0)  
Building wheels for collected packages: face-recognition-models  
  Building wheel for face-recognition-models (setup.py) ... done  
  Created wheel for face-recognition-models: filename=face_recognition_models-0.3.0-py2.py3-none-any.whl size=100566169  
  Stored in directory: /root/.cache/pip/wheels/7a/eb/cf/e9eced74122b679557f597bb7c8e4c739cfac526db1fd523d  
Successfully built face-recognition-models  
Installing collected packages: face-recognition-models, face_recognition  
Successfully installed face-recognition-models-0.3.0 face_recognition-1.3.0
```

- 필요한 패키지와 모듈 불러오기
- 구글 Colab 노트북 화면에서 실행 버튼 클릭

 # 필요한 패키지와 모듈 불러온다.

```
import cv2
import face_recognition
import pickle
import dlib

print('cv2 version :', cv2.__version__)
print('dlib version :', dlib.__version__)
```

 cv2 version : 4.6.0
dlib version : 19.24.0

■ 얼굴 인코딩에 필요한 하이퍼파라미터(Hyper parameter) 값을 설정

↻ 얼굴 인코딩에 필요한 이미지 파일 경로 등의 값을 설정

```
1 # 얼굴 인코딩에 필요한 하이퍼-파라미터 설정
2
3 dataset_paths = ['gdrive/My Drive/CV/Face Recognition/dataset/Chun/'] # 학습시킬 dataset 경로 지정
4 names = ['Chun'] # Name 지정
5 number_images = 10 # 학습시킬 이미지 수
6 image_type = '.jpg' # 이미지 확장자 지정
7 encoding_file = 'encodings.pickle' # 128개의 vector 값, 즉 특성값(feature, 숫자)으로 저장할 파일명
8
9 # Either cnn or hog. The CNN method is more accurate but slower. HOG is faster but less accurate.
10 model_method = 'cnn'
```


■ 얼굴 인코딩 특성값을 저장할 배열을 지정

- 🔄 특성값을 저장하는 배열, 특정 인물 이름을 저장하는 배열 변수명을 설정

```
1 # initialize the list of known encodings and known names
2
3 knownEncodings = []
4 knownNames = []
```

특성값을 저장할 배열 지정
name을 저장할 배열 지정

■ 특정 인물 얼굴 이미지에서 특성값을 계산하고, 그 결과를 배열에 저장

- 여기서는 총 10장의 이미지에서 특성값을 계산함
- 이미지 당 128개의 벡터 값으로 특성 값이 저장되는 것을 알 수 있음

```
1  # loop over the image paths
2  for (i, dataset_path) in enumerate(dataset_paths):
3      # extract the person name from names
4      name = names[i]
5
6      for idx in range(number_images):
7          file_name = dataset_path + str(idx+1) + image_type
8
9          image = cv2.imread(file_name)
10         # 이미지를 face_recognition 에서 처리하기 위해 RGB 형식으로 변경
11         rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
12
13         # 얼굴을 detection 하고 boxing 한다.
14         boxes = face_recognition.face_locations(rgb, model=model_method)
15
16         # 얼굴 영역(boxing Area)만 encoding(특성값)을 계산한다.
17         encodings = face_recognition.face_encodings(rgb, boxes)
18
19         # loop over the encodings
20         for encoding in encodings:
21             print(file_name, name, encoding.shape, encoding[0:2])
22             knownEncodings.append(encoding) # encoding(특성값)을 knownEncodings 배열에 저장
23             knownNames.append(name)        # name 을 knownNames 배열에 저장
```

- 각 사진 속에서 검출된 얼굴의 특성 값이 128 크기의 벡터로 계산되어 나타내어지고 있음을 확인할 수 있음
- 여기서 벡터 내의 각 수치의 의미는 알 수 없음
- 다만 동일한 인물이나 비슷한 인물의 사진을 입력하면 face_recognition.face_encodings() 함수를 통해 얻은 결과는 kNN(k-Nearest Neighbors) 알고리즘에 의해 서리 비슷한 수치 값들이 128 크기의 벡터에 저장됨

```

1 # knownEncodings 배열 : encoding(특성값)
2
3 import numpy as np
4
5 print(np.array(knownEncodings).shape) # (10, 128)
6 print(np.array(knownEncodings)[0,:]) # 첫번째 이미지 특성값 출력

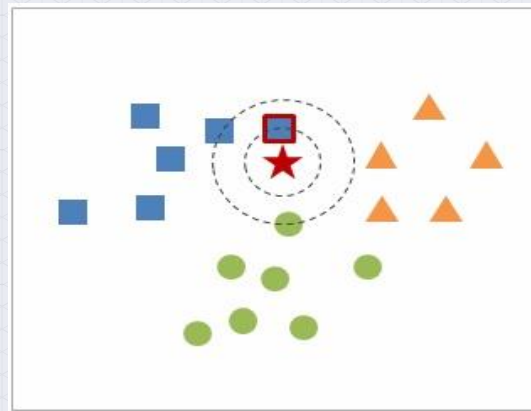
```

(10, 128)

```

[-0.09229942  0.03761727  0.0389517  -0.01408569 -0.11893666 -0.04296163
 -0.02862475 -0.09126651  0.1302492  -0.136141  0.16108797 -0.04563347
 -0.20622247 -0.08739801 -0.085503  0.15205184 -0.15774134 -0.13990714
 -0.05035019 -0.00143872  0.09135978  0.03835174  0.01379618 -0.05379851
 -0.05704164 -0.38159704 -0.10577583 -0.05868853 -0.01996204 -0.04913521
 -0.03498504  0.10400848 -0.17031103 -0.08585772  0.00770638  0.09675372
  0.01378377 -0.0424266  0.16537699  0.01374661 -0.25501731 -0.01648122
  0.05126925  0.24238919  0.1358026  0.02915912 -0.0200779 -0.13080323
  0.1001433 -0.17419206  0.03079278  0.11761235  0.09813562  0.0302905
  0.01498112 -0.04760741  0.10297292  0.09403937 -0.19751999 -0.04012734
  0.14391324 -0.10793913 -0.05710856 -0.05150723  0.17628026  0.07252432
 -0.03975128 -0.17408653  0.12144931 -0.1764311 -0.09191512  0.0606965
 -0.0804313 -0.17453927 -0.29059491 -0.01253504  0.37180018  0.14620435
 -0.16211545  0.03877455 -0.03036392 -0.0143353  0.21439359  0.10431002
 -0.0573099  0.08544192 -0.12094741  0.00546108  0.20904131 -0.02302563
 -0.06869844  0.18634835  0.03773041  0.06341273  0.08954532  0.00686301
 -0.01243788  0.03873602 -0.1836317 -0.01022412  0.10953911 -0.06280994
 -0.04130162  0.09070587 -0.12510082  0.10474179 -0.03918144  0.00527665
  0.03064633 -0.11650962 -0.09388351 -0.02239166  0.13738495 -0.18850376
  0.11356711  0.14697406  0.01392272  0.13400608  0.13681641  0.06062232
 -0.05418234 -0.1001198 -0.27050662 -0.0422022  0.14997393 -0.01974767
  0.11343588 -0.01601369]

```



k-Nearest Neighbors Algorithm

■ 얼굴 특성값(encoding)과 name을 딕셔너리(dictionary) 타입으로 저장

🔄 특성값을 딕셔너리 타입으로 저장하여 'encodings.pickle' 파일로 저장

```
▶ # encoding(특성값) 과 name 을 Dictionary 타입으로 저장한다.  
data = {"encodings": knownEncodings, "names": knownNames}  
f = open(encoding_file, "wb")  
f.write(pickle.dumps(data))  
f.close()
```

딕셔너리의 키는 "encodings", "names"로 저장된 것을 볼 수 있음

🔄 VM에서 'encodings.pickle' 파일 이름을 확인 후 내 구글 드라이브로 이동

```
[8] # 실행 결과 'encodings.pickle' 파일 확인  
!!s  
  
encodings.pickle  gdrive  sample_data  
  
[9] # 내 구글 드라이브로 파일 이동  
!mv encodings.pickle  gdrive/MyDrive/CV/Face# Recognition/  
print("encodings.pickle move complete!!!")  
  
encodings.pickle move complete!!!
```

참고

face_recognition 라이브러리의 face_locations()와 face_encodings() 함수

face_locations()와 face_encodings() 함수를 이용하여 얼굴을 검출 및 인식하고,
이를 통해서 인식한 얼굴의 이름을 저장함

face_locations() 함수

얼굴을 검출하고 얼굴의 좌표값을 저장

face_encodings() 함수

face_locations() 함수로 얻어낸 얼굴의 좌표값을 통해 해당 얼굴을
얻어내고, 얼굴 특성값(encoding)을 구하여 128개의 값으로 만듦

- 이렇게 검출된 특성값(encoding)을 훈련시켜 새로운 이미지에서 사람들의 얼굴값과 비교
→ 두 특징 점 사이의 거리가 작은(가장 비슷한) 값을 저장함
- 이 값이 설정한 적정수준 이하라면(0에 가까운 숫자일수록 비슷한 얼굴이라는 뜻) 인식한
얼굴의 이름 값을 반환함